

Knowledge Based Engineering (KBE) AE4204

Dr.ir. G. La Rocca

Flight Performance & Propulsion Group (FPP)

Lecture 5

- *Programming styles in KBE models*
- *Key KBE system features*



Content of this lecture

- **Programming styles in KBE models**
 - *Declarative* programming style
 - *Functional* programming style
- **Key KBE system features**
 - Caching and dependency tracking
 - Lazy evaluation (a.k.a. demand driven evaluation)
 - CAD kernel interaction
- **Pythonic vs KBE classes**
- **A peek under the hood of KBE systems**

Recap on KBE language purpose

KBE languages are designed to:

- Capture, formalize, and reuse engineering knowledge
- Encode design intent, rules, constraints, and rationale
- Generate families of designs, not single solutions

Traditional programming languages are designed to:

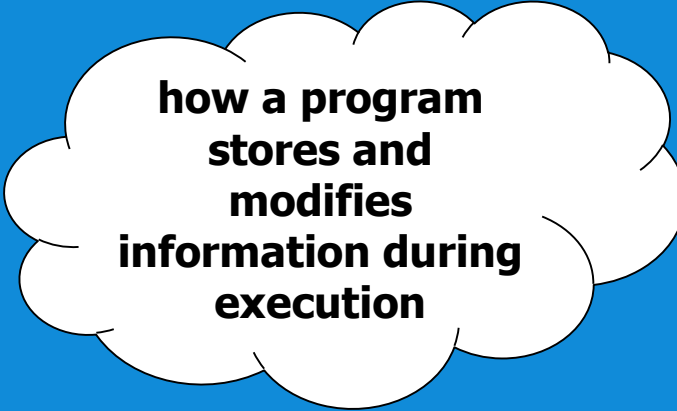
- Transform inputs → outputs
- Optimize execution efficiency
- Manage memory, control flow, and data structures

From traditional to KBE languages the **emphasis shift**

- from algorithms to knowledge structures
- From *how* to compute to *what a design is*
- From *imperative* to *declarative*

Programming styles in KBE models

- **Control of execution flow**
 - **Declarative** (*vs imperative*)
- **Handling of program state** • • •
 - **Functional** (*vs procedural*)



how a program
stores and
modifies
information during
execution

Declarative programming style

```
class Aircraft(Base):
```

```
    #: required input slot
```

```
    var1=Input()
```

```
    #: optional input slot
```

```
    var2=Input(value)
```

```
    @Attribute
```

```
    def inboard_span(self):
```

```
        return (self.var1)
```

```
    @Attribute
```

```
    def outboard_span(self):
```

```
        return (self.var2)
```

```
    @Attribute
```

```
    def span(self):
```

```
        return (self.inboard_span +  
                self.outboard_span)
```

```
    @Part
```

```
    def main_wing(self):
```

```
        return Wing(self.span)
```

=

```
class Aircraft(Base):
```

```
    #: required input slot
```

```
    var1=Input()
```

```
    #: optional input slot
```

```
    var2=Input(value)
```

```
    @Attribute
```

```
    def span(self):
```

```
        return (self.inboard_span +  
                self.outboard_span)
```

```
    @Attribute
```

```
    def inboard_span(self):
```

```
        return (self.var1)
```

```
    @Attribute
```

```
    def outboard_span(self):
```

```
        return (self.var2)
```

```
    @Part
```

```
    def main_wing(self):
```

```
        return Wing(self.span)
```

Declarative programming style

```
class Aircraft(Base):
    #: required input slot
    var1=Input()
    #: optional input slot
    var2=Input(value)
    @Attribute
    def inboard_span(self):
        return (self.var1)
    @Attribute
    def outboard_span(self):
        return (self.var2)
    @Attribute
    def span(self):
        return (self.inboard_span +
                self.outboard_span)
    @Part
    def main_wing(self):
        return Wing(self.span)
```

KBE models are typically declarative, but attribute bodies may contain **imperative** algorithms

Example of imperative code style :

```
@Attribute
def span(self):
    wingspan = 0
    wingspan = wingspan + self.inboard_span
    wingspan = wingspan + self.outboard_span
    return wingspan
```



Declarative programming style

- KBE models are naturally declarative because they describe **relationships between design parameters**.
- Declarative style fits very well the definition of **parametric design models** as used in engineering
- The developer describes **the structure of the design model**, while the KBE system determines evaluation order.
- *How is the evaluation order determined?*
 - Please, be patient, we will get there in a few slides
- *How is information stored and modified during the execution of a KBE model?*
 - Let's talk about **functional** (vs *procedural*) **coding style**

Functional programming style

```
class Aircraft(Base):
    #: required input slot
    var1=Input()
    #: optional input slot
    var2=Input(value)
    @Attribute
    def inboard_span(self):
        return (self.var1)
    @Attribute
    def outboard_span(self):
        return (self.var2)
    @Attribute
    def span(self):
        return (self.inboard_span +
                self.outboard_span)
    @Part
    def main_wing(self):
        return Wing(self.span)
```

KBE programs mostly use a **functional programming style**

I.e. the computation of **@Attribute** and **@Parts** are treated **as the evaluation of mathematical functions**:

- result depends only on inputs
- no modification of program state, i.e.
 - No modification of something else
 - **No side effects**
- same inputs → same output

Also parts behave like **functions returning objects**. If span changes, main_wing is not modified.

The part is re-evaluated and a new instance of Wing is produced

Functional programming style

```
class Aircraft(Base):  
    #: required input slot  
    var1=Input()  
    #: optional input slot  
    var2=Input(value)  
    @Attribute  
    def inboard_span(self):  
        return (self.var1)  
    @Attribute  
    def outboard_span(self):  
        return (self.var2)  
    @Attribute  
    def span(self):  
        return (self.inboard_span +  
                self.outboard_span)  
    @Part  
    def main_wing(self):  
        return Wing(self.span)
```

KBE programs mostly use a **functional programming style, however...**

Input() slots are **non-functional** elements of the KBE model:

- Their values can be explicitly modified
- Changing their values alters the state of the model and may propagate changes to dependent attributes
- Inputs are the **changing knobs** that allow to modify our objects

Non-functional behavior is called **procedural**

Functional programming style

```
class Aircraft(Base):
    #: required input slot
    var1=Input()
    #: optional input slot
    var2=Input(value)
    counter=Input(0)
    def increment_counter(self):
        self.counter = self.counter + 1
    @Attribute
    def inboard_span(self):
        return (self.var1)
    @Attribute
    def outboard_span(self):
        return (self.var2)
    @Attribute
    def span(self):
        return (self.inboard_span +
                self.outboard_span)
```

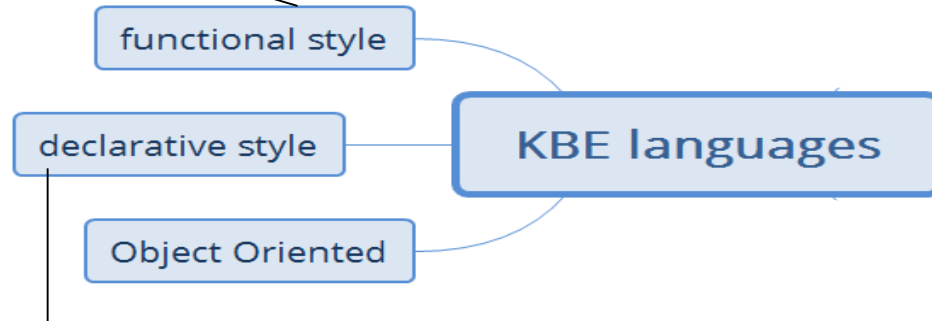
KBE programs mostly use a **functional programming style, however...**

Example of procedural behaviour in a method:

- If I call myaircraft.increment_counter(), I change the state of the object
- The value increase of counter is the **side effect**
- The result of calling this method depends on how many times it was executed, which is characteristic of **procedural programming**

Recap of KBE programming styles and system mechanisms

..but also procedural



..but also imperative

Inherited from the host language
(e.g. Python in ParaPy)

How we write KBE models

Provided by the KBE system runtime
(e.g. ParaPy)

How the KBE system execute models

- **Caching & dependency tracking**
- **Lazy Evaluation**

Caching and dependency tracking

So far we learnt that:

- KBE models are **mostly** written in a **declarative** and **functional** style
- Instead of defining a sequence of operations, **a KBE model mostly defines relations between quantities**
- Most of the elements of a KBE model are defined **as function of other elements**
- *How is the evaluation order determined?*
- *If we change one input, how does the system know which quantities must be recomputed?*
- *Should the system recompute everything, or only the quantities that depend on the changed input?*

Caching and dependency tracking

KBE systems maintain a **dependency graph** between all the elements of the model

This graph records **which attributes depend on which other attributes**.

It describes **how information flows through the model**

In our Aircraft example model we have:

Var1 → inboard_span

Var2 → outboard_span

Inboard_span+outboard_span → span

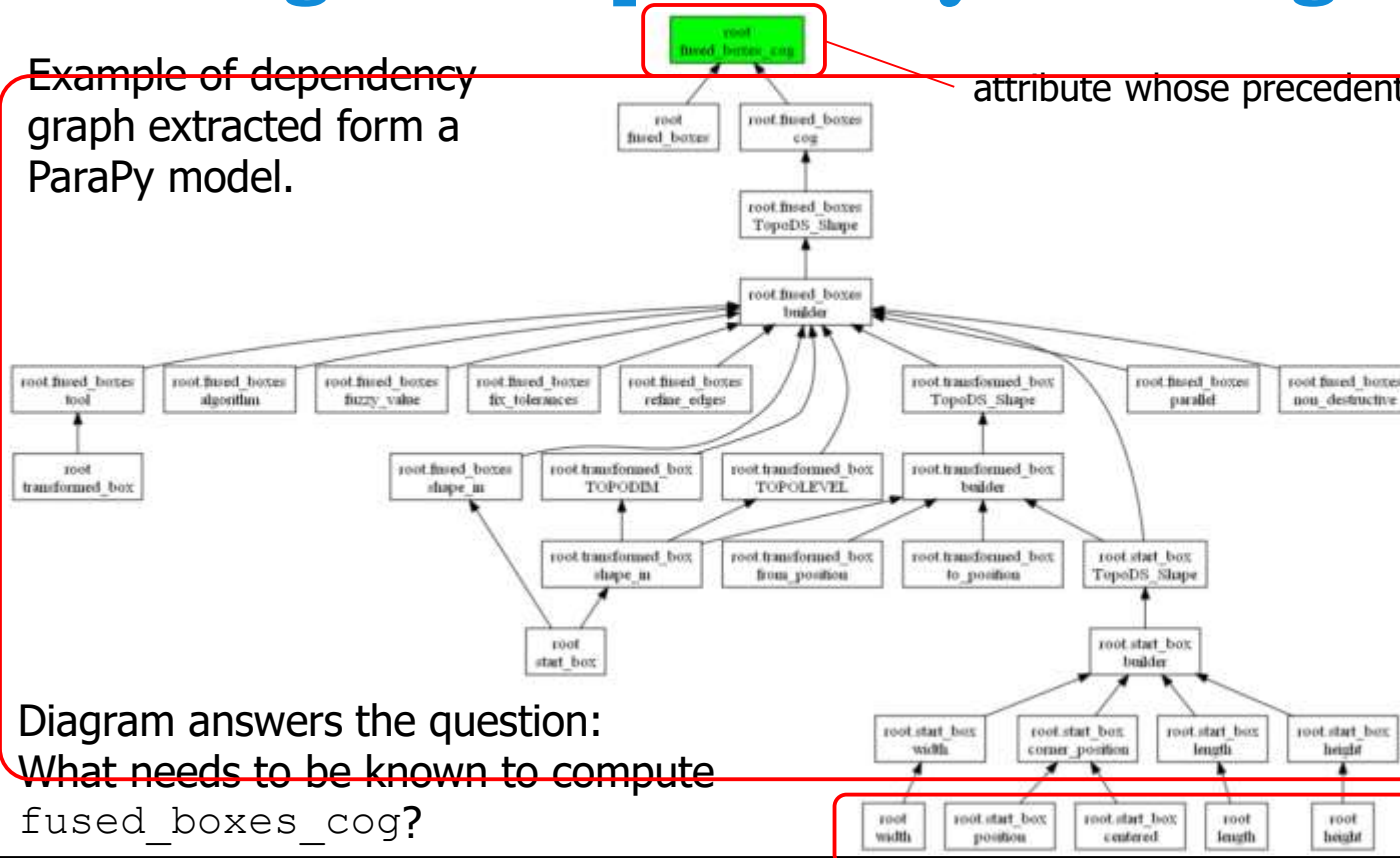
span → main_wing

"→" = affects

```
class Aircraft(Base):
    #: required input slot
    var1=Input()
    #: optional input slot
    var2=Input(value)
    @Attribute
    def inboard_span(self):
        return (self.var1)
    @Attribute
    def outboard_span(self):
        return (self.var2)
    @Attribute
    def span(self):
        return (self.inboard_span +
                self.outboard_span)
    @Part
    def main_wing(self):
        return Wing(self.span)
```

Caching and dependency tracking

Example of dependency graph extracted from a ParaPy model.



attribute whose precedents you asked for

attributes that must be evaluated to compute it

Diagram answers the question:
What needs to be known to compute fused_boxes_cog?

The actual inputs

Caching and dependency tracking

A dependency graph can be extremely large, concatenating thousands of function evaluations.

Suppose **an attribute has already been computed**.

If we (or some other element in the dependency graph) request its value again, *should the system recompute it every time?*

- KBE systems provide a mechanism called **caching** which gives the ability to memorize (i.e. to cache) the value of computed attributes, so that they can be directly retrieved from memory, **if none of its dependencies have changed**
- The **dependency tracking** mechanism keeps track of the current validity of the cached values. If dependencies have changed, **it invalidate them***

Lazy evaluation (demand driven evaluation)

A KBE system evaluates model elements only when their value is required.

Evaluation is triggered by:

- **Direct user request** (e.g., asking for the value of an attribute or visualizing an object)
- **Indirect requests** from another object (e.g. an attribute needed to compute another attribute).

Only the necessary* parts of the model are evaluated.

Caching + Dep. Tracking + Lazy evaluation

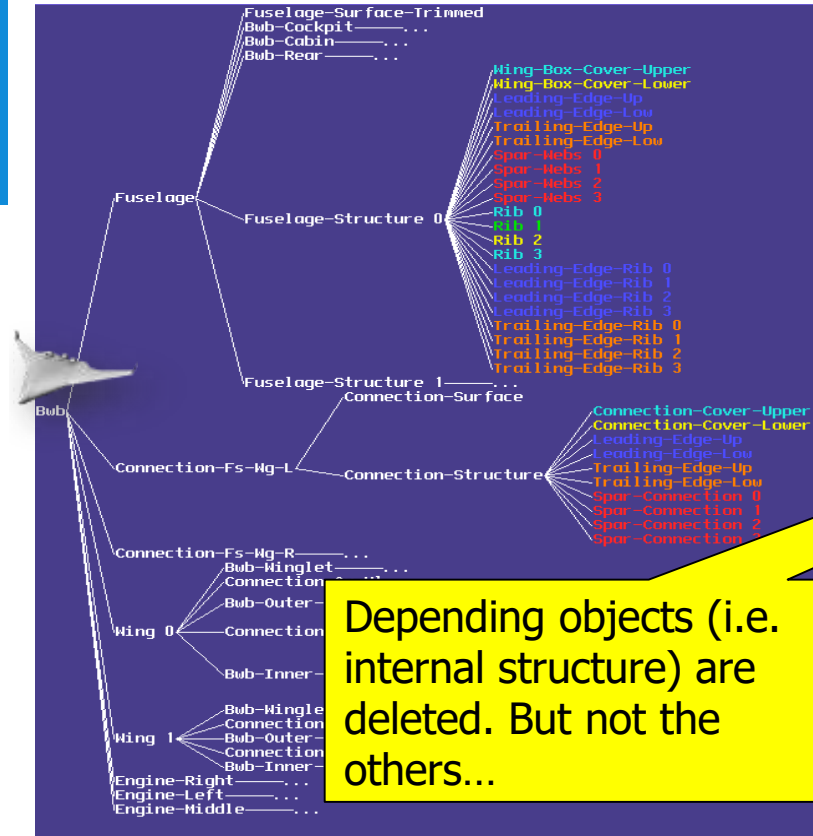
To recap:

- dependency tracking **decides what** to recompute
- caching **remembers** previous results
- lazy evaluation **decides when** to compute

Their combination:

- avoids recomputing **expensive analysis or geometry operations**
- allows evaluation of **only** the relevant branches of large engineering models
- supports **fast design iterations** when only a few input change
- supports **incremental development and prototyping**

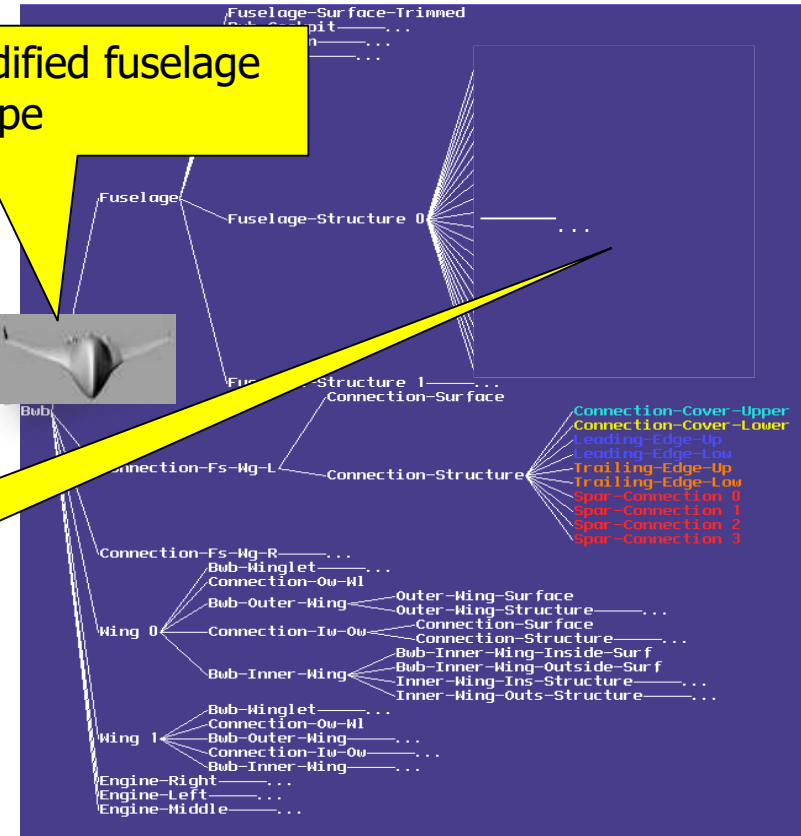
Caching and dependency tracking



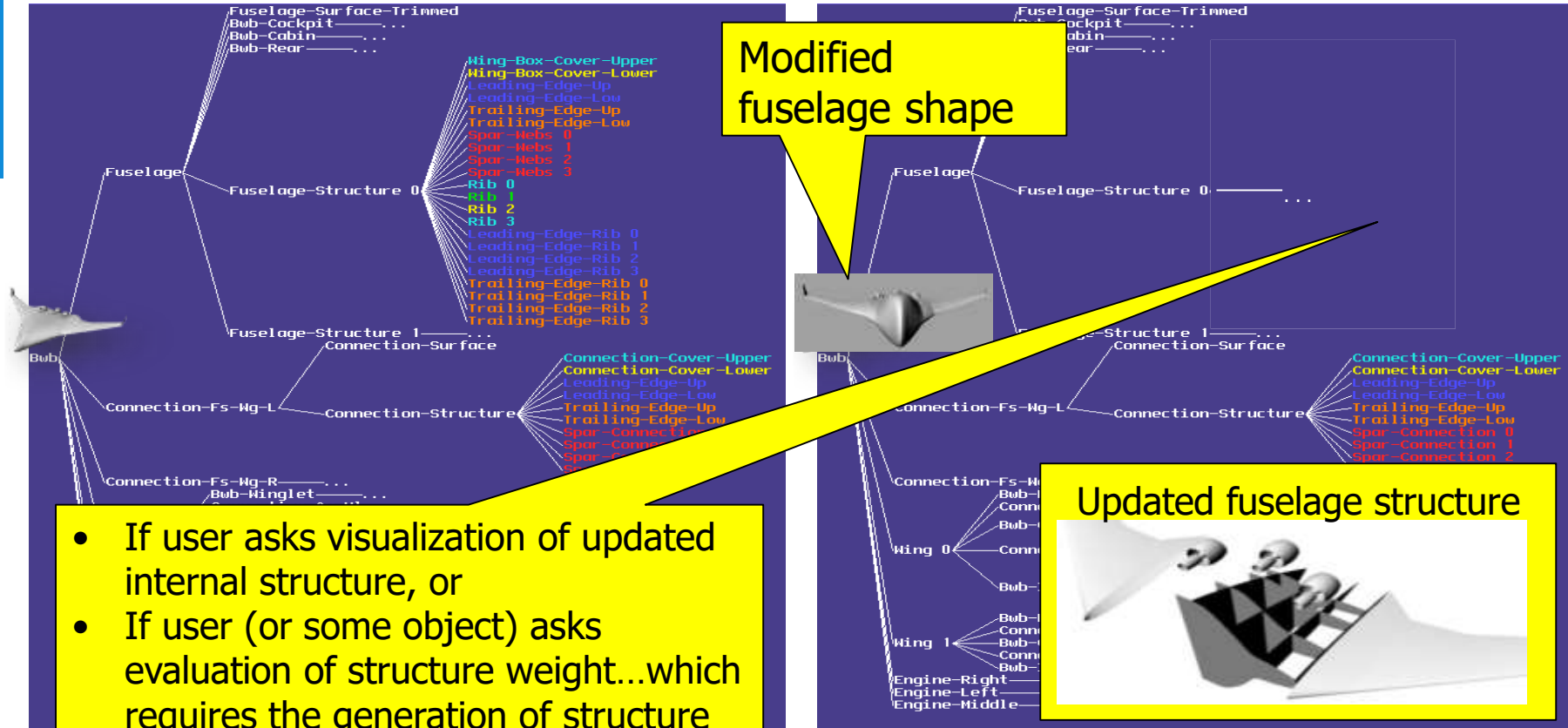
Modified fuselage shape



Depending objects (i.e. internal structure) are deleted. But not the others...

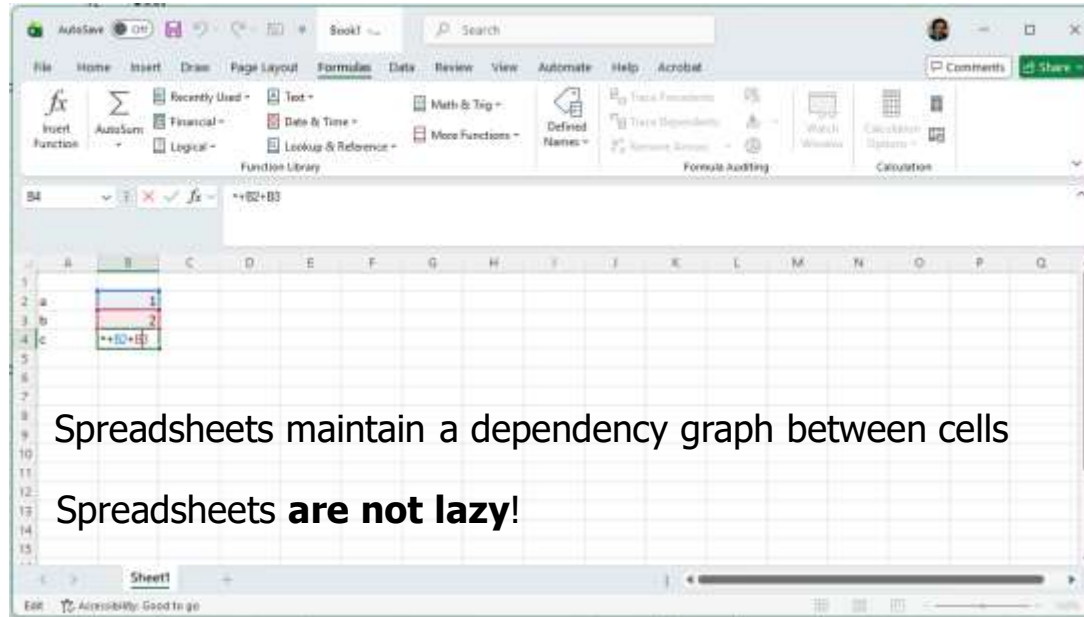


Lazy evaluation



Caching and dependency tracking

Do you know another commodity tool that uses the same caching and dependency tracking mechanism as KBE systems? Sure you do...



Spreadsheets maintain a dependency graph between cells

Spreadsheets **are not lazy!**

Python class *vs* Parapy class

- Caching & dependency
- Lazy approach

(see Tutorial material)

Typical Python Class Layout

```
class MyClass(Superclass):                                # inherit from Superclass  
  
    foo = 1                                                # class attribute  
  
    def bar(self):                                          # methods...  
        return self.foo + 1  
  
    def quz(self):  
        return Box(width=self.bar)  
  
    def qux(self, spam):  
        print(spam)
```

Python classes

```
class Wing():  
    taper = 0.2  
    c_root = 4  
  
    def get_c_tip(self):  
        print("I take 10 minutes to return")  
        return self.c_root * self.taper
```

Class variables

Class method

```
>>> wing = Wing()  
>>> wing.get_c_tip()  
"I take 10 minutes to return"  
0.8  
>>> wing.get_c_tip()  
"I take 10 minutes to return"  
0.8  
>>> wing.taper = 0.3  
>>> wing.get_c_tip()  
"I take 10 minutes to return"  
1.2
```

non-caching

Class initialization method

```
class Wing():  
  
    def __init__(self, taper=0.2, c_root=4):  
        self.taper = taper  
        self.c_root = c_root  
        self.c_tip = self.get_c_tip()  
  
    def get_c_tip(self):  
        print("I take 10 minutes to return")  
        return self.c_root * self.taper
```

```
>>> wing = Wing()  
"I take 10 minutes to return"  
>>> wing.c_tip  
0.8  
>>> wing.c_tip  
0.8  
>>> wing.taper = 0.3  
>>> wing.c_tip  
0.8  
>>> wing.c_tip = wing.get_c_tip()  
"I take 10 minutes to return"  
>>> wing.c_tip  
1.2
```

caching

non-lazy

no dependency
tracking

non-lazy

Typical Python Class Layout

```
class MyClass(Superclass):                                # inherit from Superclass  
  
    foo = 1                                                # class attribute  
  
    def bar(self):                                         # methods...  
        return self.foo + 1  
  
    def quz(self):  
        return Box(width=self.bar)  
  
    def qux(self, spam):  
        print(spam)
```

Typical ParaPy Class Layout

```
class MyClass(Base, Superclasses):
```

inherit from Base

```
    foo = Input(1)
```

Input

```
    @Attribute
```

Attribute

decorator

```
    def bar(self):
```

```
        return self.foo + 1
```

decorator

```
    @Part
```

Part

```
    def quz(self):
```

```
        return Box(width=self.bar)
```

```
    def qux(self, spam):
```

method

```
        print(spam)
```

ParaPy adds dependency tracking, caching and lazy evaluation

Python Class

```
class Wing():
    taper = 0.2
    c_root = 4

    def get_c_tip(self):
        print("I take 10 minutes to return")
        return self.c_root * self.taper
```

```
>>> wing = Wing()
>>> wing.get_c_tip()
"I take 10 minutes to return"
0.8
>>> wing.get_c_tip()
"I take 10 minutes to return"
0.8
>>> wing.taper = 0.3
>>> wing.get_c_tip()
"I take 10 minutes to return"
1.2
```

ParaPy Class

```
class Wing(Base):
    taper = Input(0.2)
    c_root = Input(4)

    @Attribute
    def c_tip(self):
        print("I take 10 minutes to return")
        return self.c_root * self.taper
```

```
>>> wing = Wing()
>>> wing.c_tip
"I take 10 minutes to return"
0.8
>>> wing.c_tip
0.8
>>> wing.taper = 0.3
>>> wing.c_tip
"I take 10 minutes to return"
1.2
```

lazy

caching

lazy

dependency tracking



Under the hood of a KBE system:

Implementation of the caching & dependency tracking mechanism

Runtime caching and dependency tracking

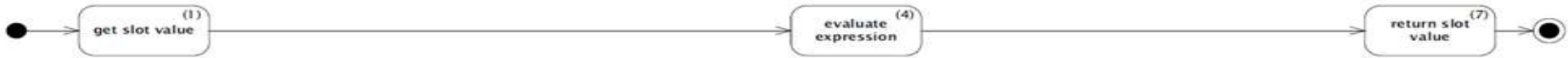
Let assume, the following slots (Input slots) have been assigned, using a general-purpose programming language:

length = 2

width = 3

Let now request the evaluation of the following slot (get slot value):

height = length + width



height?



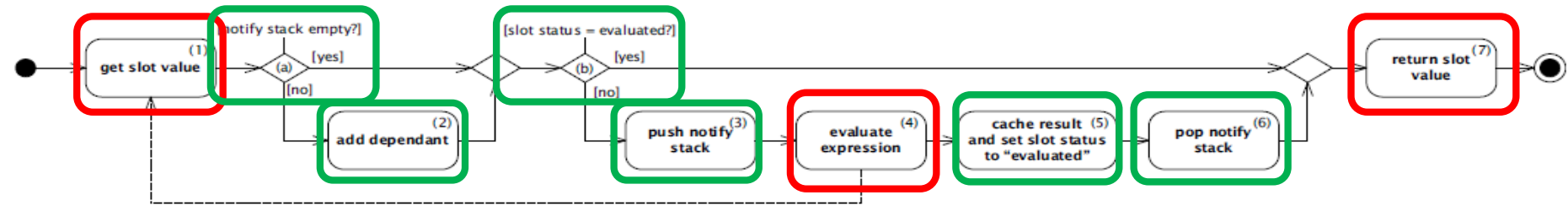
height: 5

Let request again the evaluation of the slot height

Why wasting time to re-evaluate this expression when length and width have not changed?

Runtime caching and dependency tracking

In a KBE system a special bookkeeping mechanism tracks dependencies between model elements and avoids recomputing expressions unless one of their inputs has changed

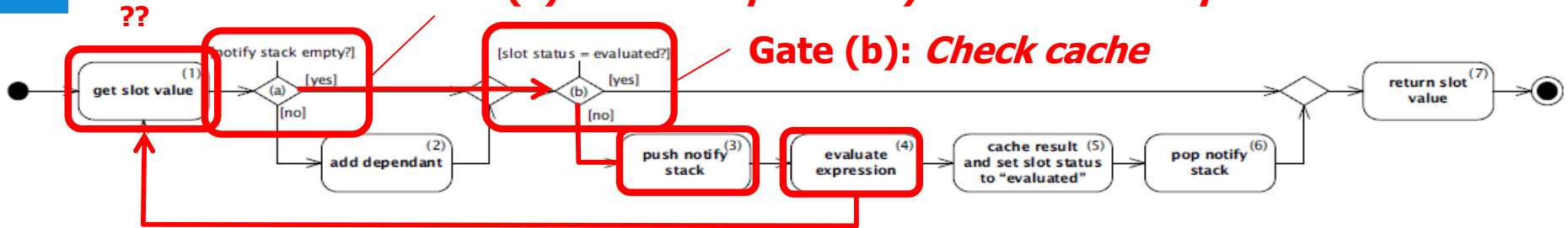


The extra **steps (2, 3, 5, 6)** and the two **decision gates (a) and (b)**, together, enable the ***caching & dependency tracking mechanism***

Runtime caching and dependency tracking

Gate (a): Check dependency stack or "Notify stack"

Gate (b): Check cache



```
class Sample(Base):
```

```
#: input slots
length=Input(2)
width=Input(3)
```

```
@Attribute
```

```
def height(self):
```

```
    return (self.length +
            self.width)
```

Notify stack (check dependency)

- height

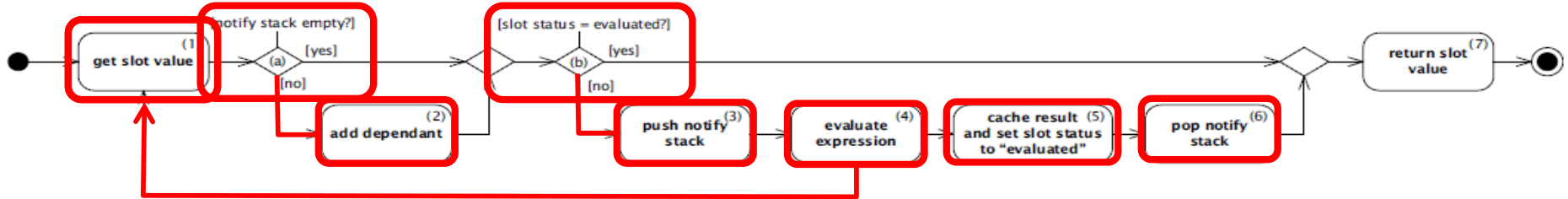
Cache

slot	initial		
	status	cache	dependants
length	:unbound	nil	nil
width	:unbound	nil	nil
height	:unbound	nil	nil

Action: Instantiate the class *Sample* and ask the value of *height*

Runtime caching and dependency tracking

???



Notify stack (check dependency)

- height

Cache

slot	status	cache	dependants
length	evaluated	2	(height)
width	unbound	nil	nil
height	unbound	nil	nil

```
class Sample(Base):
```

```
    #: input slots
```

```
    length=Input(2)
```

```
    width=Input(3)
```

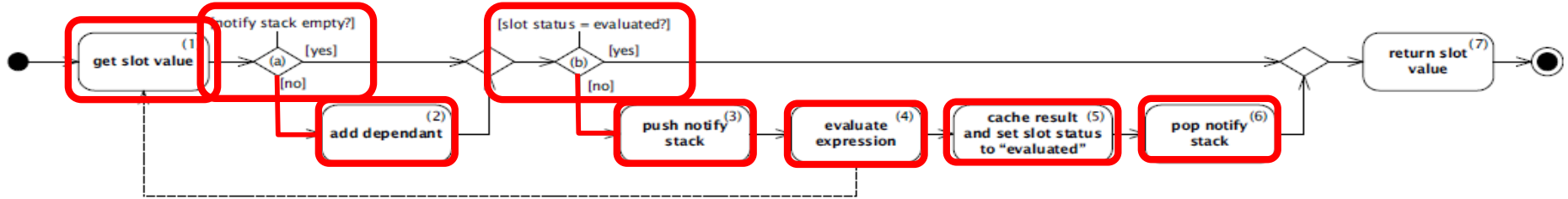
```
@Attribute
```

```
def height(self):
```

```
    return (self.length +
            self.width)
```

Runtime caching and dependency tracking

width?



```
class Sample(Base):
    #: input slots
    length=Input(2)
    width=Input(3)

    @Attribute
    def height(self):
        return (self.length +
                self.width)
```

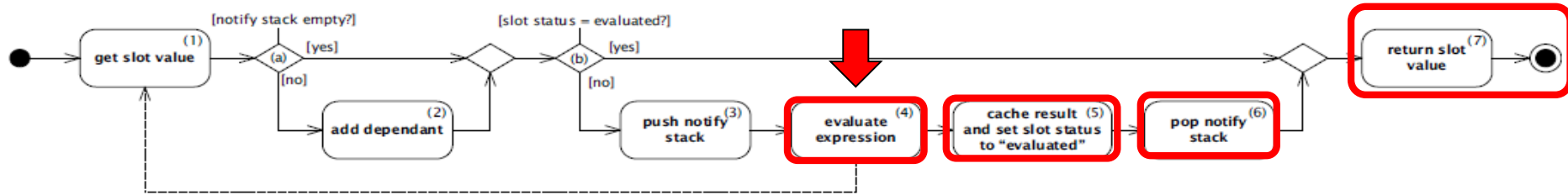
Notify stack (check dependency)

- height

Cache

slot	status	cache	dependants
length	evaluated	2	(height)
width	evaluated	3	(height)
height	unbound	nil	nil

Runtime caching and dependency tracking



Notify stack (check dependency)



Cache

slot	status	cache	dependants
length	evaluated	2	(height)
width	evaluated	3	(height)
height	evaluated	5	nil

```
class Sample(Base):
```

```
    #: input slots
```

```
    length=Input(2)
```

```
    width=Input(3)
```

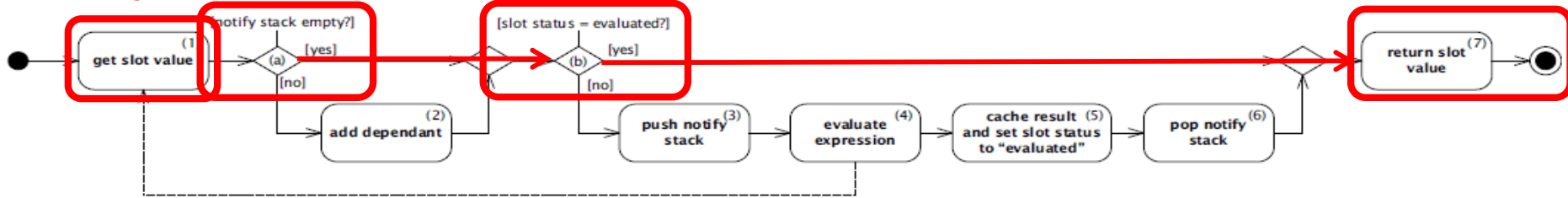
```
@Attribute
```

```
def height(self):
```

```
    return (self.length +
            self.width)
```

Runtime caching and dependency tracking

Height?



Notify stack (check dependency)



Cache

slot	after get		
	status	cache	dependants
length	evaluated	2	(height)
width	evaluated	3	(height)
height	evaluated	5	nil

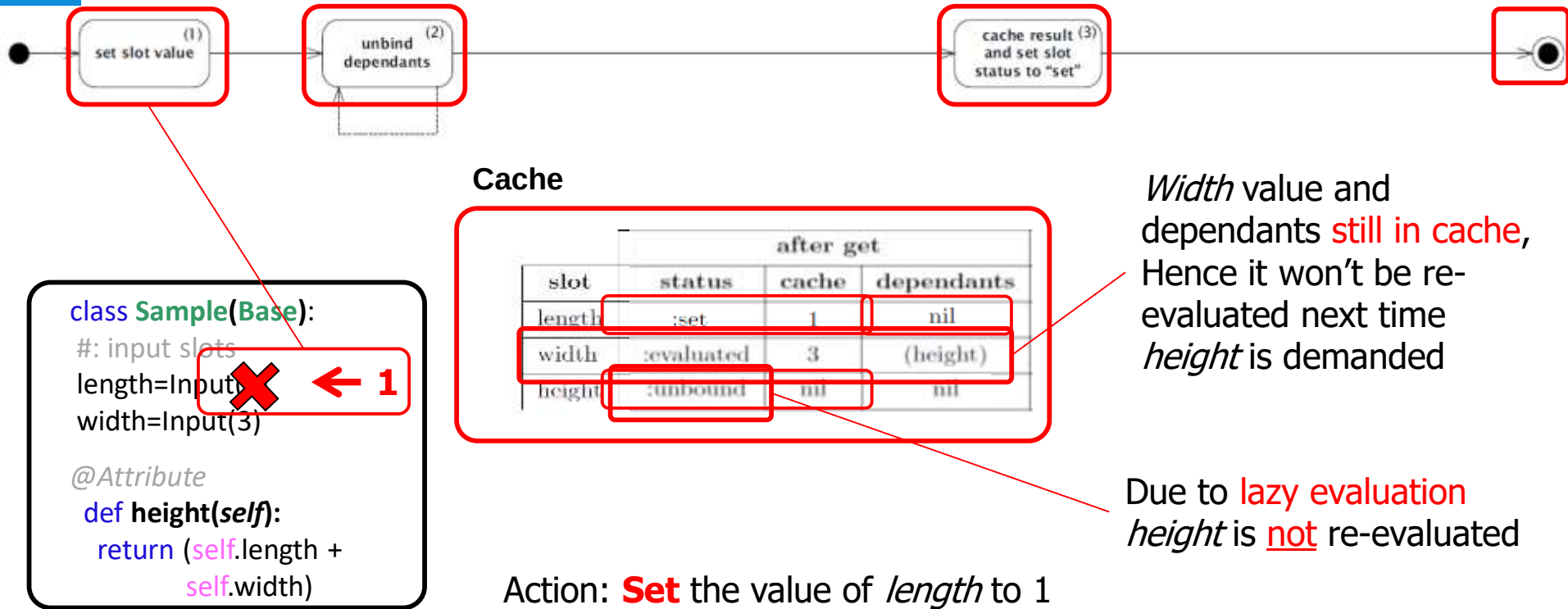


Action: ask **again** the value of *height*

```
class Sample(Base):
    #: input slots
    length=Input(2)
    width=Input(3)

    @Attribute
    def height(self):
        return (self.length +
                self.width)
```

Runtime caching and dependency tracking





Under the hood of a KBE system:

How the KBE system interacts with the CAD kernel

KBE systems and link to CAD geometry kernels

- A **geometry kernel** is the computational engine running underneath a CAD system.
 - It consists of a **collection of numerical algorithms and libraries** used to create and manipulate **curves, surfaces, and solid models**.
 - It performs the actual geometric computations, such as intersections, Boolean operations, offsets, fillets, and so on.
- KBE systems allow engineers to control these operations **programmatically**, which enables automated generation of complex parametric geometries.

KBE systems and link to CAD geometry kernels

- Many general-purpose languages can be used to integrate and control an external geometry kernel **through an API**
- However, the way a KBE system interacts with a geometry kernel is distinctive: geometry is generated through a **declarative and functional model**, where geometric operations are **automatically** managed through **dependency tracking, caching, and lazy evaluation**
- **ParaPy** uses the industry grade open source geometry kernel **Open Cascade**



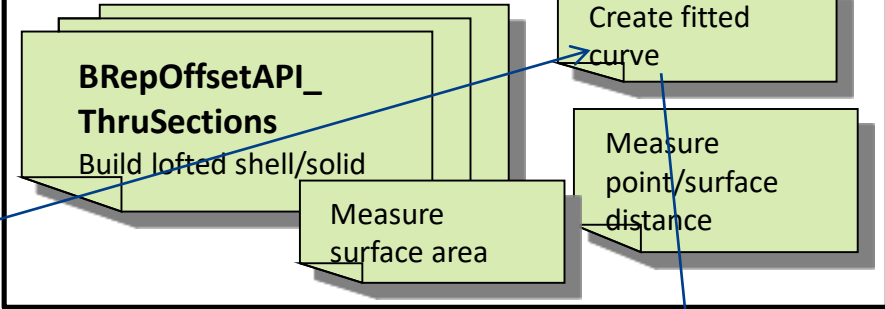
KBE language

```
@Part
def airfoil_root(self):
    return FittedCurve(self.list_of_root_points)
```

```
@Part
def airfoil_tip(self):
    return FittedCurve(self.list_of_tip_points)
```

```
@Part
def wing(self):
    return LoftedSurface(profiles = [self.airfoil_root, self.airfoil_tip])
```

OpenCascade kernel (C/C++)



airfoil_root:FittedCurve	
curve	binding()
points	list_of_root_points
...	...

1

Pointer to list_of_root_points

3

Pointer to fitted curve 1

4



Garbage Collector

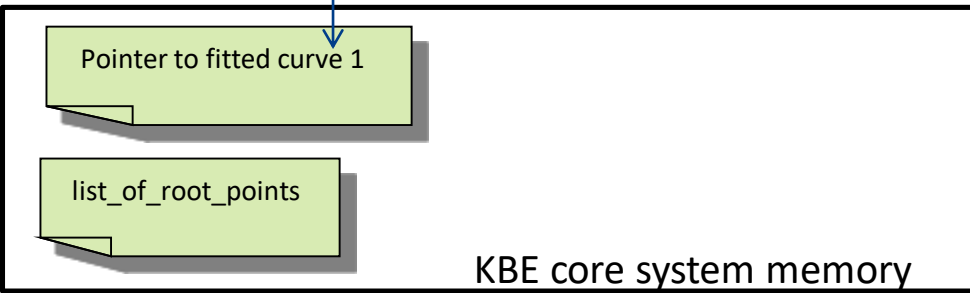
Store pointer to fitted curve 1

2

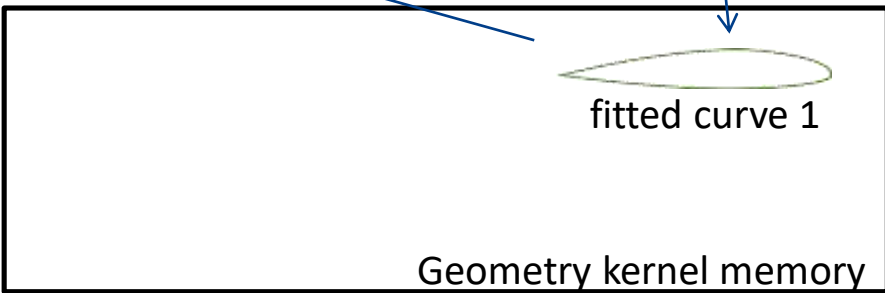
Create fitted curve



Finalizer functions



KBE core system memory



Geometry kernel memory

KBE language

```
@Part
def airfoil_root(self):
    return FittedCurve(self.list_of_root_points)

@Part
def airfoil_tip(self):
    return FittedCurve(self.list_of_tip_points)

@Part
def wing(self):
    return LoftedSurface(profiles = [self.airfoil_root, self.airfoil_tip])
```

wing:LoftedSurface	
Surface	binding()
Area	binding()
...	...



KBE core system memory

Pointer to fitted curve 1

Pointer to fitted curve 2

Pointer to lofted surface

OpenCascade kernel (C/C++)

BRepOffsetAPI_ThruSections

Build lofted shell/solid

Measure surface area

Create fitted curve

Measure point/surface distance

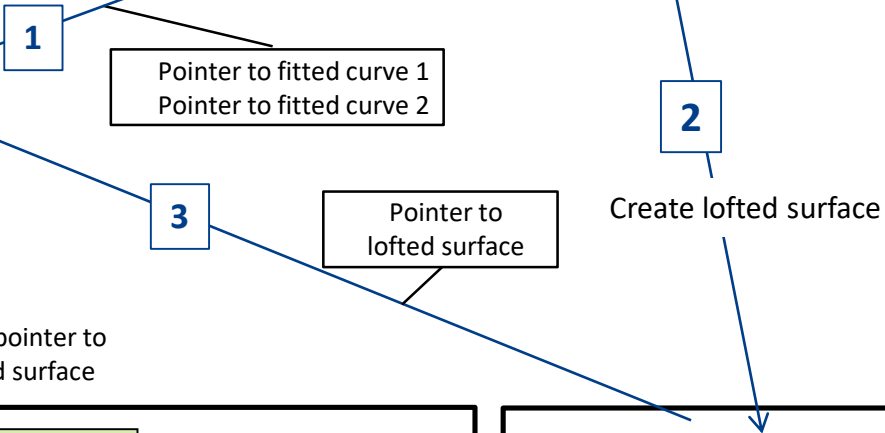
Geometry kernel memory

Lofted surface

fitted curve 1

fitted curve 2

Finalizer functions



KBE language

```
@Part
def airfoil_root(self):
    return FittedCurve(self.list_of_root_points)

@Part
def airfoil_tip(self):
    return FittedCurve(self.list_of_tip_points)

@Part
def wing(self):
    return LoftedSurface(profiles = [self.airfoil_root, self.airfoil_tip])
```

wing:LoftedSurface	
Surface	binding ()
Area	binding ()
...	...



Garbage Collector

1

Pointer to lofted surface

2

Measure surface area

4

Cache lofted surface area value

3

Pass lofted surface area value

Pointer to fitted curve 1

Pointer to fitted curve 2

Pointer to lofted surface

lofted surface area

KBE core system memory

OpenCascade kernel (C/C++)

BRepOffsetAPI_ThruSections

Build lofted shell/solid

Measure surface area

Create fitted curve

Measure point/surface distance

Lofted surface

Fitted curve 2

Fitted curve 1

Finalizer functions

Geometry kernel memory

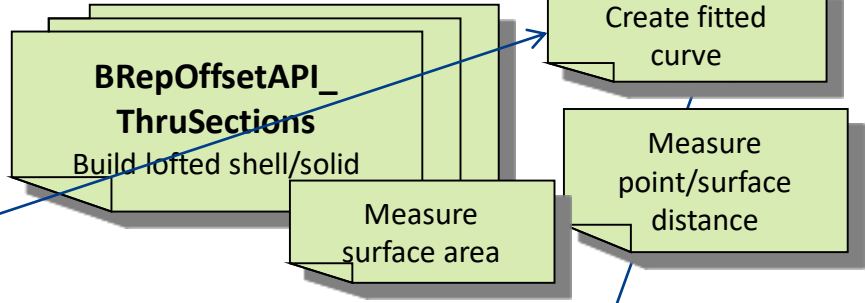
KBE language

```
@Part
def airfoil_root(self):
    return FittedCurve(self.list_of_root_points)

@Part
def airfoil_tip(self):
    return FittedCurve(self.list_of_tip_points)

@Part
def wing(self):
    return LoftedSurface(profiles = [self.airfoil_root, self.airfoil_tip])
```

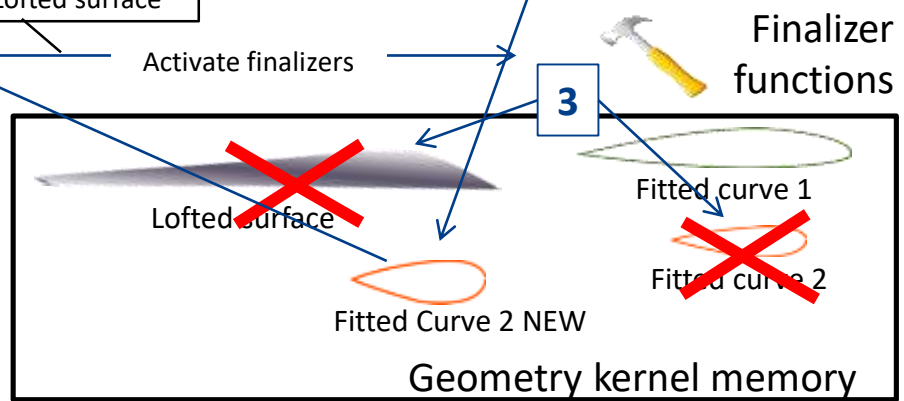
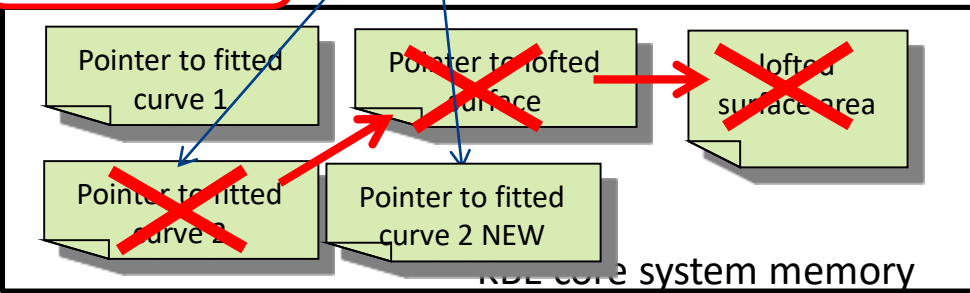
OpenCascade kernel (C/C++)



airfoil tip:FittedCurve	
curve	binding()
points	list_of_tip_points
...	...



Garbage Collector

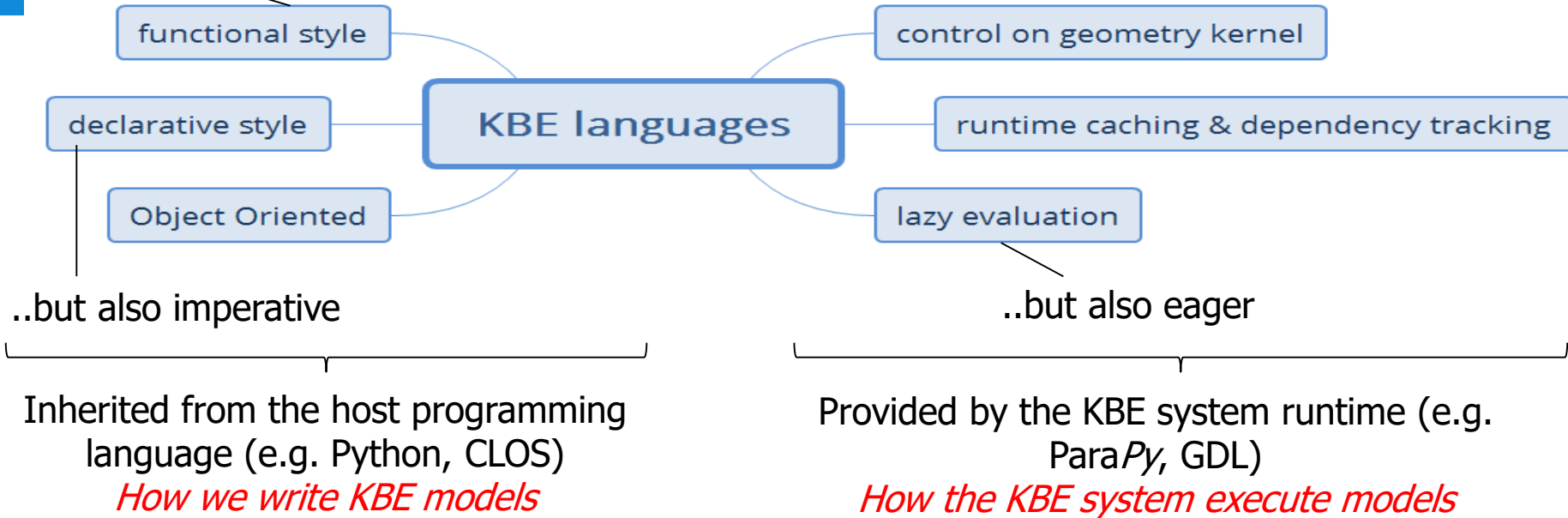


KBE core system memory

Geometry kernel memory

Summary of KBE programming styles and system mechanisms

..but also procedural



Main reference material for this lecture

1. **Book** *MDO supported by KBE* by Wiley: **Chapter 9 on KBE**
2. **Journal paper** *Knowledge based engineering: Between AI and CAD. Review of a language based technology to support engineering design*, G. La Rocca,, *Journal of Advanced Engineering Informatics*, Vol. 26, pp.159–179, 2012.
3. **ParaPy documentation** and **Training Material**
4. Tutorial exercises

